

Redis

Base de datos en memoria — tipos de datos, casos de uso, variantes y configuración en Next.js

Redis es una base de datos **en memoria** (in-memory) extremadamente rápida. En lugar de guardar los datos en disco como PostgreSQL o MySQL, los guarda en la RAM del servidor.

```
PostgreSQL → datos en disco → lectura en milisegundos
Redis       → datos en RAM   → lectura en microsegundos (100-1000x más rápido)
```

¿Para qué se usa?

1. Caché — guardar resultados de consultas costosas

```
// Sin caché — consulta a BD en cada request
const productos = await prisma.producto.findMany() // 200ms

// Con Redis — segunda vez instantáneo
const cache = await redis.get('productos')
if (cache) return JSON.parse(cache) // 1ms

const productos = await prisma.producto.findMany()
await redis.setex('productos', 3600, JSON.stringify(productos)) // 1 hora
return productos
```

2. Sesiones — guardar y invalidar sesiones de usuario

```
// Guardar sesión
await redis.setex(`session:${sessionId}`, 86400, JSON.stringify(usuario))

// Recuperar sesión
const session = await redis.get(`session:${sessionId}`)

// Invalidar sesión (logout real e instantáneo)
await redis.del(`session:${sessionId}`)
```

3. Rate limiting — limitar requests por usuario

```
const key = `ratelimit:${ip}`
const requests = await redis.incr(key)

if (requests === 1) {
  await redis.expire(key, 60) // ventana de 60 segundos
}

if (requests > 100) {
  return Response.json({ error: 'Demasiadas peticiones' }, { status: 429 })
}
```

4. Cola de tareas — procesar trabajos en background

```
// Añadir tarea a la cola
await redis.lpush('emails:queue', JSON.stringify({
  to: 'usuario@email.com',
```

```

    subject: 'Bienvenido',
    template: 'welcome'
  )))

// Worker que procesa la cola
while (true) {
  const tarea = await redis.brpop('emails:queue', 0)
  await enviarEmail(JSON.parse(tarea))
}

```

5. Pub/Sub — comunicación en tiempo real

```

// Publisher — publica un mensaje
await redis.publish('notificaciones', JSON.stringify({
  userId: '123',
  mensaje: 'Tienes un nuevo pedido'
}))

// Subscriber — escucha mensajes
redis.subscribe('notificaciones', (mensaje) => {
  const data = JSON.parse(mensaje)
  enviarNotificacionAlUsuario(data)
})

```

Tipos de datos en Redis

Redis no es solo clave-valor simple. Tiene varios tipos de datos nativos:

String — el más básico

```

await redis.set('nombre', 'Ana')
await redis.get('nombre') // → 'Ana'
await redis.setex('token', 3600, 'abc123') // expira en 1 hora

```

Hash — objeto con campos

```

await redis.hset('usuario:123', {
  nombre: 'Ana', email: 'ana@email.com', rol: 'admin'
})
await redis.hget('usuario:123', 'nombre') // → 'Ana'
await redis.hgetall('usuario:123') // → { nombre, email, rol }

```

List — lista ordenada (cola o pila)

```

await redis.lpush('cola', 'tarea1', 'tarea2') // añade al inicio
await redis.rpush('cola', 'tarea3') // añade al final
await redis.lpop('cola') // saca del inicio
await redis.lrange('cola', 0, -1) // todos los elementos

```

Set — conjunto sin duplicados

```

await redis.sadd('tags', 'react', 'typescript', 'nextjs')
await redis.smembers('tags') // → ['react', 'typescript', 'nextjs']
await redis.sismember('tags', 'react') // → true

```

Sorted Set — conjunto con puntuación (ranking)

```

await redis.zadd('ranking', 1500, 'Ana')
await redis.zadd('ranking', 2300, 'Juan')
await redis.zadd('ranking', 1800, 'María')

// Top 3
await redis.zrevrange('ranking', 0, 2, 'WITHSCORES')
// → ['Juan', 2300, 'María', 1800, 'Ana', 1500]

```

Tipos / variantes de Redis

Redis Open Source

El original, self-hosted. Lo instalas en tu propio servidor.

```
docker run -d -p 6379:6379 redis:alpine
```

Upstash Redis (recomendado para Next.js)

El más popular en proyectos Next.js/Vercel. Serverless, pago por uso, plan gratuito generoso. Se integra perfectamente con Edge Runtime.

```
import { Redis } from '@upstash/redis'

const redis = new Redis({
  url: process.env.UPSTASH_REDIS_URL!,
  token: process.env.UPSTASH_REDIS_TOKEN!,
})

await redis.set('key', 'value')
await redis.get('key')
```

Vercel KV

Basado en Upstash pero integrado directamente en Vercel. Sin configuración extra.

```
import { kv } from '@vercel/kv'

await kv.set('key', 'value')
await kv.get('key')
```

Otras variantes

- **Redis Cloud** — Redis como servicio gestionado, plan gratuito de 30MB
- **KeyDB** — Fork de Redis más rápido, multithreaded, compatible con Redis
- **Dragonfly** — Alternativa moderna, mucho más rápida en benchmarks

Comparativa de variantes

Variante	Hosting	Precio	Ideal para
Redis OSS	Self-hosted	Gratis	Control total, producción propia
Redis Cloud	Gestionado	Freemium	Apps de cualquier tamaño
Upstash	Serverless	Pay per use	Next.js, Vercel, Edge Runtime
Vercel KV	Serverless	Pay per use	Proyectos desplegados en Vercel
KeyDB	Self-hosted	Gratis	Alto rendimiento
Dragonfly	Self-hosted	Gratis	Máximo rendimiento

Configuración en Next.js con Upstash

```
npm install @upstash/redis

// lib/redis.ts
import { Redis } from '@upstash/redis'

export const redis = new Redis({
  url: process.env.UPSTASH_REDIS_REST_URL!,
  token: process.env.UPSTASH_REDIS_REST_TOKEN!,
})

// Usar en Server Component o API Route
```

```
import { redis } from '@lib/redis'  
const data = await redis.get('mi-clave')
```

¿Cuándo añadir Redis a tu proyecto?

App lenta por consultas repetidas	→ Caché con Redis
Necesitas logout real y rápido	→ Sesiones en Redis
Demasiados requests de un usuario	→ Rate limiting con Redis
Tareas pesadas (emails, PDFs)	→ Cola de tareas con Redis
Notificaciones en tiempo real	→ Pub/Sub con Redis